

Projet de synthèse — Linux

Application web de gestion des accès SSH à une machine virtuelle

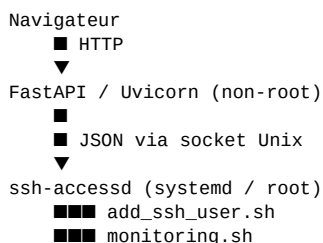
Rapport technique : robustesse, sécurité et supervision

Objectif

Mettre en place une application web hébergée sur une VM permettant à un utilisateur de demander un accès SSH en fournissant un login et une clé publique. La partie web ne possède aucun privilège root. Toutes les opérations nécessitant ces privilèges sont réalisées par des scripts Bash, orchestrés par un daemon unique exécuté comme service systemd.

1. Architecture

L'architecture sépare la couche web non privilégiée de la couche système privilégiée. FastAPI communique avec le daemon par un socket Unix. Le daemon est le seul composant autorisé à lancer les scripts Bash système.



2. Application web FastAPI

L'application est développée en Python avec FastAPI et Jinja2. Elle possède une page de demande d'accès SSH et une page de monitoring.

Route	Rôle
GET /	Affiche le formulaire de demande d'accès.
POST /request-access	Transmet le login et la clé publique au daemon.
GET /monitoring	Affiche le dashboard de supervision.
GET /api/monitoring	Demande les métriques au daemon et retourne le JSON.

3. Socket Unix

Le fichier de socket utilisé est `/run/ssh-access/ssh-access.sock`. FastAPI y envoie des requêtes JSON. Le socket est configuré avec le groupe `sshaccess` et des permissions `0660`. L'utilisateur exécutant l'application web appartient à ce groupe.

Cette organisation évite de donner des privilèges root à FastAPI : l'application peut demander une opération au daemon, mais ne l'exécute pas elle-même.

4. Daemon systemd

Le fichier `daemon/ssh_access_daemon.py` constitue le chef d'orchestre. Il écoute le socket Unix, decode les requêtes JSON, vérifie l'action demandée, exécute le script correspondant et renvoie le résultat.

Le service systemd utilise notamment un redémarrage automatique en cas d'échec (`Restart=on-failure`). Le répertoire d'exécution du socket est géré par systemd.

```
RuntimeDirectory=ssh-access
RuntimeDirectoryMode=0750
User=root
Group=sshaccess
Restart=on-failure
RestartSec=3
```

5. Création des comptes SSH

Le script `scripts/add_ssh_user.sh` réalise les opérations nécessitant root. Il est exécuté uniquement par le daemon.

Contrôles et mesures :

- utilisation de `set -euo pipefail` ;
- vérification du nombre d'arguments ;
- validation du format du login ;
- validation du type de clé publique SSH ;
- refus si le compte existe déjà ;
- création du répertoire personnel et de `~/.ssh` ;
- permissions 700 sur `~/.ssh` et 600 sur `authorized_keys` ;
- attribution des fichiers au nouvel utilisateur.

6. Monitoring

Le script `scripts/monitoring.sh` collecte les informations système et produit un document JSON. Le daemon exécute ce script puis transmet les données à FastAPI.

Élément	Données affichées
CPU	Pourcentage d'utilisation processeur.
RAM	Total, utilisée, libre et pourcentage utilisé.
Disque	Total, utilisé, disponible et pourcentage utilisé sur <code>/</code> .
Système	Hostname, uptime et load average.
Utilisateurs	Login, UID, répertoire personnel et shell.
Processus	Utilisateur, PID, commande, CPU et RAM des principaux processus.

7. Actualisation sans rechargement

La page `monitoring.html` interroge `/api/monitoring` avec JavaScript. Les métriques sont actualisées automatiquement toutes les cinq secondes et peuvent également être actualisées avec le bouton prévu à cet effet. Le DOM est mis à jour sans recharger la page.

8. Robustesse

- séparation des privilèges entre FastAPI et le daemon ;
- validation des entrées avant les opérations système ;
- liste explicite des actions acceptées par le daemon ;
- timeout de 10 secondes pour l'exécution des scripts ;
- gestion des erreurs de permission, socket absent, connexion refusée et JSON invalide ;
- redémarrage automatique du daemon par `systemd` ;
- retour structuré des résultats sous forme JSON.

9. Sécurité

- FastAPI fonctionne sans droits root.
- Un seul daemon possède la responsabilité des opérations privilégiées.
- Le canal de communication privilégié est un socket Unix, non un port réseau dédié.
- L'accès au socket est limité par le groupe système sshaccess et le mode 0660.
- La clé privée SSH n'est jamais demandée ou stockée par l'application.
- Le fichier authorized_keys reçoit des permissions restrictives.
- Les entrées utilisateur sont contrôlées côté Bash avant création du compte.

10. Arborescence

```
ssh-access/  
├── app/  
│   ├── main.py  
│   ├── templates/  
│   │   ├── index.html  
│   │   └── monitoring.html  
│   ├── static/  
│   │   └── style.css  
├── daemon/  
│   └── ssh_access_daemon.py  
└── scripts/  
    ├── add_ssh_user.sh  
    └── monitoring.sh
```

11. Flux de création d'un accès

1. L'utilisateur saisit son login et sa clé publique.
2. FastAPI reçoit la requête HTTP.
3. FastAPI construit une requête JSON.
4. La requête est envoyée au socket Unix.
5. Le daemon reçoit et analyse l'action.
6. Le daemon lance add_ssh_user.sh avec les privilèges root.
7. Le script valide les données et crée le compte SSH.
8. Le résultat revient au daemon puis à FastAPI.
9. L'utilisateur reçoit le résultat dans l'interface web.
10. La connexion SSH est ensuite possible avec la clé privée correspondante.

12. Supervision GCP

La VM est hébergée sur Google Cloud Platform. Le monitoring applicatif fournit une visibilité immédiate sur les ressources de la VM. Une intégration avec Google Cloud Monitoring peut compléter ce dispositif avec des métriques persistantes, des dashboards d'infrastructure et des alertes.

13. Améliorations possibles

- mettre en place un protocole de tramage explicite pour les messages du socket ;
- durcir davantage le service systemd avec des mécanismes de sandboxing adaptés aux fichiers réellement nécessaires ;
- utiliser jq pour générer le JSON du script de monitoring ;
- ajouter une agrégation CPU/RAM par utilisateur ;
- ajouter une authentification applicative si le service est utilisé hors environnement pédagogique ;
- placer l'application derrière HTTPS et un reverse proxy en production ;
- ajouter des alertes et de l'historisation dans Google Cloud Monitoring.

14. Conclusion

L'architecture mise en place respecte la contrainte principale du projet : la partie web reste non privilégiée, tandis que les opérations root sont centralisées dans un daemon unique exécuté par systemd et réalisées par des scripts Bash. La communication utilise un socket Unix avec des permissions contrôlées. Le dashboard de monitoring permet en outre de suivre les ressources de la VM, les utilisateurs et les processus sans rechargement de la page.

État du projet : synthèse au 17 septembre 2026.